Python for Complete Beginners

A Classroom-Ready 1-Hour Teaching Plan

Write and run your first Python programs in 60 minutes

Audience: First-time programmers • **Duration:** 60 minutes • **Format:** Interactive, hands-on

How to Use This Plan

This document is your complete script for the session. Everything is timed and sequenced so you can present it top-to-bottom without preparation. Along the way you will find:

- **Blue code boxes** — type these live and let students copy them.
- **Green output boxes** — the exact result students should see on screen.
- **Yellow "Try It" boxes** — short exercises to run every few minutes.
- **Red "Watch Out" boxes** — common beginner mistakes and their fixes.

> **Before the session — 2-minute setup**
> The easiest zero-install option is to open **https://www.python.org** → or use an online editor like **replit.com**, **Google Colab**, or **programiz.com/python-programming/online-compiler**. Ask students to open one editor tab so they can code along.

Contents

1. The 60-Minute Syllabus at a Glance

Keep a timer visible. Each block ends with a tiny hands-on task so nobody just watches.

Time	Topic	What Students Do
0–5 min	**Welcome + What is Python & running code**	Watch demo, run first print
5–12 min	**print() & Comments**	Print their own messages
12–22 min	**Variables & Data Types**	Create variables, guess types
22–30 min	**User Input & Type Conversion**	Build a greeting program
30–40 min	**Operators & Expressions**	Mini calculator exercise
40–50 min	**Decisions: if / elif / else**	Odd-or-even, grade checker
50–57 min	**Loops: for & while**	Count & repeat exercises
57–60 min	**Revision + Final Challenge**	Recap quiz, take-home problem

Golden rule for the hour

For beginners, **typing and running code beats listening.** Aim for the class to run at least **8–10 small programs** themselves. If you must cut something, keep the exercises and trim your talking.

2. Teaching Material — Topic by Topic

2.1 What Is Python & Running Your First Line (0–5 min)

Simple explanation: Python is a language for giving instructions to a computer. You write instructions in plain-looking text, and Python reads them from top to bottom and does exactly what you say. It is popular because it reads almost like English.

Say this to the class: "A program is just a to-do list for the computer. Today we learn how to write that list."

Your very first program

```
print("Hello, world!")
```

Output

```
Hello, world!
```

Explain the pieces: `print` is a **command** that shows text on screen; the **parentheses** hold what to show; the **quotes** mark the text.

> **Try It (60 seconds)**
> Have everyone change the message and run it, e.g. `print("My name is Aisha")`. Celebrate — they just ran a program!

2.2 print() and Comments (5–12 min)

print() displays whatever you give it. You can print text, numbers, or several items separated by commas.

```
print("Learning Python is fun")
print(2026)
print("Score:", 95)
print("Line one")
print("Line two")
```

Output

```
Learning Python is fun
2026
Score: 95
Line one
Line two
```

Comments are notes for humans. Python ignores any line starting with `#`. Use them to explain your code.

```
# This line is ignored by Python
print("Comments help you remember things")  # note at end of line
```

Output

```
Comments help you remember things
```

Try It (2 min)

Ask students to print **three lines** about themselves (name, city, favourite food) and add one comment explaining what the program does.

2.3 Variables & Data Types (12–22 min)

Metaphor: A variable is a labelled box that stores a value so you can use it later. You put something in the box with `=` and use the box by its name.

```
name = "Ravi"
age = 21
height = 5.9
is_student = True

print(name)
print("Age:", age)
```

Output

```
Ravi
Age: 21
```

The four core data types beginners need

- **String (str)** — text, always in quotes: `"Ravi"`, `"Hello"`
- **Integer (int)** — whole numbers: `21`, `-4`, `100`
- **Float** — numbers with a decimal point: `5.9`, `3.14`
- **Boolean (bool)** — only two values: `True` or `False`

Use `type()` to ask Python what kind of value something is:

```
print(type("Ravi"))
print(type(21))
print(type(5.9))
print(type(True))
```

Output

```
<class 'str'>
<class 'int'>
<class 'float'>
<class 'bool'>
```

Try It — Guess the type (2 min)

Show these on screen and let students shout the type before you reveal it: `"42"`, `42`, `3.0`, `False`. (Answers: str, int, float, bool.)

⚠ Watch Out

Variable names cannot start with a number or contain spaces. Use `first_name`, not `first name` or `1name`. Names are also **case-sensitive**: `Age` and `age` are different boxes.

2.4 User Input & Type Conversion (22–30 min)

input() pauses the program and lets the user type something. Whatever they type comes back as **text (a string)** — even if they type numbers.

```
name = input("What is your name? ")
print("Nice to meet you,", name)
```

Sample run

```
What is your name? Meera
Nice to meet you, Meera
```

Converting types: Because input is always text, convert it before doing maths. Use `int()` for whole numbers and `float()` for decimals.

```
age = input("Enter your age: ")
age = int(age)            # turn text "20" into number 20
print("Next year you will be", age + 1)
```

Sample run

```
Enter your age: 20
Next year you will be 21
```

⚠ Watch Out — the #1 beginner error here

Doing maths on input without converting causes an error or wrong result. `age + 1` fails if `age` is still text. Always wrap number inputs with `int(...)` or `float(...)` first.

Try It — Greeting machine (3 min)

Ask the user for their name and birth year, then print: **"Hi <name>, you are about <age> years old."** Hint: `age = 2026 - int(year)`.

2.5 Operators & Expressions (30–40 min)

Operators let you calculate and compare. Python works like a calculator.

Arithmetic operators

```
print(10 + 3)   # addition  -> 13
print(10 - 3)   # subtraction -> 7
print(10 * 3)   # multiplication -> 30
print(10 / 3)   # division -> 3.333...
print(10 // 3)  # whole-number division -> 3
print(10 % 3)   # remainder (modulo) -> 1
```

```
print(10 ** 2)  # power -> 100
```

Output

```
13
7
30
3.3333333333333335
3
1
100
```

The star of the show: `%` (modulo) gives the remainder. `10 % 3` is 1. This is how we test even/odd later: an even number has `number % 2 == 0`.

Comparison operators — these give True or False

```
print(5 > 3)    # True
print(5 < 3)    # False
print(5 == 5)   # equal? -> True
print(5 != 4)   # not equal? -> True
print(5 >= 5)   # True
```

Output

```
True
False
True
True
True
```

> ⚠ **Watch Out — = vs ==**
> `=` stores a value (`x = 5`). `==` asks "are these equal?" (`x == 5`). Mixing them up is the most common beginner bug.

> **Try It — Mini calculator (3 min)**
> Ask the user for two numbers and print their sum, difference and product. Hint: `a = int(input("First: "))`.

2.6 Making Decisions: if / elif / else (40–50 min)

Idea: Programs choose different paths depending on a condition — just like real life: "IF it rains, take an umbrella, ELSE wear sunglasses."

> **Indentation matters!**
> Python uses **indentation (spaces)** to know which lines belong inside the `if`. Use a consistent 4 spaces. The line before an indented block always ends with a colon `:`.

```
age = int(input("Enter your age: "))

if age >= 18:
```

```
    print("You are an adult.")
else:
    print("You are a minor.")
```

Sample run

```
Enter your age: 16
You are a minor.
```

Choosing between many options with elif

```
marks = int(input("Enter marks: "))

if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
elif marks >= 50:
    print("Grade C")
else:
    print("Fail")
```

Sample run

```
Enter marks: 82
Grade B
```

> ⚠ **Watch Out — indentation & colons**
> Forgetting the `:` or mixing tabs and spaces causes `IndentationError` or `SyntaxError`. Pick 4 spaces and stay consistent for every indented line.

> ✎ **Try It — Odd or Even (3 min)**
> Ask for a number and print whether it is **odd** or **even**. Hint: `if number % 2 == 0:`.

2.7 Loops: Doing Things Repeatedly (50–57 min)

Idea: A loop repeats instructions so you don't copy-paste. Two everyday loops:

for loop — repeat a set number of times

```
for i in range(5):
    print("Hello", i)
```

Output

```
Hello 0
Hello 1
Hello 2
Hello 3
Hello 4
```

`range(5)` produces the numbers **0, 1, 2, 3, 4** (it starts at 0 and stops before 5).

while loop — repeat while a condition is true

```
count = 1
while count <= 3:
    print("Count is", count)
    count = count + 1
```

Output

```
Count is 1
Count is 2
Count is 3
```

⚠ Watch Out — infinite loops & off-by-one

In a `while` loop you MUST change the condition variable (e.g. `count = count + 1`), or it runs forever. Also remember `range(5)` gives 0–4, **not** 1–5.

Try It — Countdown (2 min)

Print the numbers **1 to 10** using a `for` loop. Hint: `for i in range(1, 11):`.

3. Common Beginner Mistakes — Quick Reference

Keep this table on screen while students work. Most first-hour errors are on this list.

✗ Common Mistake	☑ How to Fix It
`Forgetting quotes: print(Hello)`	`print("Hello")`
`Using = instead of ==`	Use == to compare, = to assign
`Maths on raw input: age + 1`	`Convert first: int(input(...))`
`Missing colon: if x > 5`	`if x > 5:`
`Wrong indentation inside if/loop`	Use exactly 4 spaces, be consistent
`Mismatched brackets: print("Hi"`	`Close every ( and " you open`
`Expecting range(5) to give 1–5`	`It gives 0,1,2,3,4 — use range(1,6)`
`while loop never ends`	Update the counter inside the loop
`Case error: Print / PRINT`	`Python is case-sensitive: use print`

Teaching tip

When an error appears, don't hide it — read the **last line of the red error message** aloud. It usually names the problem (e.g. `SyntaxError`, `NameError`). Teaching students to read errors calmly is one of the most valuable skills of the hour.

4. Quick Revision — Everything in One Page (57–60 min)

Read each line aloud and ask the class to complete it. This cements the hour.

Cheat sheet

```
# 1. Show something on screen
print("Hello")

# 2. Comment (Python ignores it)
# this is a note

# 3. Variables & types
name = "Ravi"      # str
age  = 21          # int
pi   = 3.14        # float
ok   = True        # bool

# 4. Input (always text) + convert
x = int(input("Number: "))

# 5. Operators
#  + - * /   //  %   **      (maths)
#  == != > < >= <=          (compare -> True/False)

# 6. Decisions
if x > 0:
    print("positive")
elif x == 0:
    print("zero")
else:
    print("negative")

# 7. Loops
for i in range(3):
    print(i)

count = 0
while count < 3:
    count = count + 1
```

One-line recap of each topic

- **print()** shows output; # writes comments.
- **Variables** store values; the 4 types are str, int, float, bool.
- **input()** reads text — convert with `int()` / `float()` for maths.
- **Operators** calculate (`+ - * / % **`) and compare (`== != > <`).
- **if / elif / else** choose a path; remember the colon and 4-space indent.
- **for / while** repeat instructions; `range()` starts at 0.

☑ 30-second exit quiz (ask aloud)

- What symbol starts a comment? **(#)**
- Which converts text to a whole number? **(int())**
- What does `7 % 2` give? **(1)**
- = or == to compare two values? **(==)**
- How many times does `range(4)` loop? **(4: 0,1,2,3)**

5. Final Challenge — Solve It Yourself 🎯

Give this to students to complete independently after the session. It uses every concept from the hour: input, conversion, operators, conditions, and (optionally) a loop.

🧩 Problem: Simple Grade & Ticket Calculator

Part A — Grade checker. Ask the user for their marks (0–100) and print their grade using this scale: 90+ = A, 75–89 = B, 50–74 = C, below 50 = Fail.

Part B — Even/Odd check. Ask the user for any number and print whether it is even or odd.

Part C (bonus) — Countdown. Using a loop, print a countdown from 5 down to 1, then print "Go!".

Expected sample run

Sample run

```
Enter your marks: 82
Grade: B
Enter a number: 7
7 is odd
5 4 3 2 1 Go!
```

Model solution (for the instructor — do not hand out first)

```
# Part A - Grade checker
marks = int(input("Enter your marks: "))
if marks >= 90:
    print("Grade: A")
elif marks >= 75:
    print("Grade: B")
elif marks >= 50:
    print("Grade: C")
else:
    print("Grade: Fail")

# Part B - Even or odd
number = int(input("Enter a number: "))
if number % 2 == 0:
    print(number, "is even")
else:
    print(number, "is odd")

# Part C - Countdown (bonus)
for i in range(5, 0, -1):
    print(i, end=" ")
print("Go!")
```

Output

```
Enter your marks: 82
Grade: B
Enter a number: 7
7 is odd
5 4 3 2 1 Go!
```

Encourage & extend

Fast finishers can add: a calculator that keeps asking until the user types "quit", or a program that prints the multiplication table of a number using a loop. Remind everyone: **errors are normal — every programmer reads and fixes them daily.**